

Verifica automatica di programmi

Riccardo Torre

5 marzo 2026

Indice

1	Alcune note iniziali	1
2	Una panoramica sulla verifica di programmi	1
Lezione del giorno: 2 marzo 2026		

1 Alcune note iniziali

Il primo esame è composto da 2 prove parziali, ciascuna vale 25% prova iniziale + 25% prova finale. Il restante 50% è riservato ad un progetto individuale

$$25\%PI + 25\%PF + 50\%P$$

Gli altri appelli (2,3,4 e 5) sono 100% esame scritto (no progetto).

Il corso finisce circa a metà maggio. Il primo appello è all'inizio di giugno. Il progetto viene assegnato circa a metà del semestre e comprende una relazione. Il libro di testo è di *Aanon Brandley & Zohan Manna* che usa un linguaggio di programmazione ideale che si chiama π che è simile al C ma semplificato. **Rust** sarà presente in questo corso. Verus (della Microsoft) e Why (Rust Belt). La verifica Rust (collegli di Parigi che insegnano codifica Rust).

2 Una panoramica sulla verifica di programmi

Floyd e Hoare furono i pionieri di questa teoria alla fine degli anni 60. Il limite fondamentale per la verifica automatica di programmi resta sempre lo stesso: non possiamo creare un verificatore universale che decida la correttezza semantica per programmi arbitrari[itf]. Questa condizione viene garantita dal teorema 1.

Teorema 1 (di Rice). *Il teorema di Rice stabilisce che tutte le proprietà non banali delle funzioni calcolabili da programmi (tranne quelle sempre vere o sempre false) sono indecidibili, ovvero non esiste un algoritmo generale che, dato un programma, possa determinare automaticamente se soddisfa tale proprietà.[Wik]*

In pratica, si aggirano questi limiti con approssimazioni, analisi statiche conservative, o restrizioni sul dominio (es. programmi finiti o lineari), ma la verifica totale resta impossibile[itf]. Esistono degli strumenti per effettuare quella che viene chiamata *analisi statica* per fare la dimostrazione di correttezza di un programma a partire dal suo codice sorgente in modo quasi totalmente meccanico. Ci sono tre tecniche fondamentali che andremo a studiare in questa prima parte del corso:

1. **La specifica:** specificazione di un programma. Quello che un programma deve fare (caricare qualcosa su un sito, ordinare un array) viene descritto con formule in **logica del 1° ordine**. Ad esempio per dire che un array è ordinato tra l'indice l e l'indice u bisogna dire

$$\text{sorted}(a, l, u) \equiv \forall i \forall j: l \leq i \leq j \leq u \implies a[i] \leq a[j]$$

La logica composizionale da sola non è sufficientemente espressiva. Avremo bisogno di scrivere per ogni funzione¹ di un programma bisogna specificare:

- (a) **Pre-condizione:** è un qualcosa che si assume vero prima che inizi la computazione della funzione. Contiene anche ciò che la funzione assume per poter lavorare. Descrive lo stato del programma prima dell'esecuzione della funzione. Utilizzeremo la parola chiave “*requires*” cioè ciò che la funzione richiede prima della sua esecuzione.
- (b) **Post-condizione:** ciò che vale dopo l'esecuzione della funzione. Descrive lo stato del programma dopo la computazione della funzione. Utilizzeremo la parola chiave “*ensures*” cioè ciò che la funzione assicura dopo la sua esecuzione.

L'esecuzione di una funzione modifica lo stato di un programma.

Le annotazioni (la categoria generale a cui appartengono le pre-condizioni e le post-condizioni) sono formule incorporate nel codice e si distinguono nel modo seguente:

- **pre-condizioni:** formule che devono valere prima dell'esecuzione della funzione;
- **post-condizioni:** formule che devono valere dopo l'esecuzione della funzione;
- **asserzioni:** formule contenute nel corpo della funzione che devono valere in quel punto nel codice.

Le annotazioni sono riconoscibili dentro il codice perché riportano la chiocciola @L²: F³
Quindi da un programma

```
<type> foo(<type_1> param_1 ... <type_n> param_n)
{
    <block_1>
    <block_2>
    return( )
}
```

si ottiene un programma annotato

```
@pre: F
@post: G
<type> foo(<type_1> param_1 ... <type_n> param_n)
{
    <block_1>
    @L: H
    <block_2>
    return( )
}
```

¹ogni programma può essere visto come una collezione di funzioni.

²L è una label.

³formula di logica del primo ordine che deve essere vera nello stato del programma.

C'è una branca dell'informatica che studia come generare automaticamente le annotazioni. In questo corso non riusciamo a vedere questa parte e le scriveremo sempre manualmente. La formula H deve essere vera nello **stato del programma** a quel punto (linea interessata da $@L: H$).

Stato del programma È definito dall'indirizzo di programma nel PC⁴ e le variabili del programma (ad esempio $x : int, y : int, i : int, j : int, a : int[]$). Lo stato è una fotografia della memoria nell'esecuzione.

```
PC = 0010011
x = 5
y = 0
j = -2
a = [0, 0, 3, 0, 17, -1]
```

Quindi lo stato del programma può essere visto come un assegnamento di valori del tipo richiesto al PC e a ogni variabile (parametri) del programma.

```
@pre: F
@post: G
<type> foo(<type_1> param_1, <type_n> param_n)
{
    <block_1>
    @L: H
    <block_2>
    return( )
}
```

Se la funzione `foo` viene chiamata con `foo(a, 0, 16)` vuol dire che $param_1 = a$, $param_2 = 0$, $param_3 = 16$. I **parametri** (o “*parametri formali*”) sono quelli che definiscono la funzione, gli **argomenti** (o “*parametri attuali*”) sono quelli della chiamata di funzione. Un parametro può essere passato **per valore** (la funzione non può modificarlo; tutti i parametri sono passati per valore nel linguaggio `pi`) o **per riferimento** (il valore può essere modificato dalla funzione).

2. **Metodo delle asserzioni induttive:** È un metodo per dimostrare **proprietà di correttezza parziale** (o in generale “*safety*”) ovvero che il programma non va in stato d'errore. Se un'asserzione è vera nello stato 0, e nello stato n , allora deve essere anche nello stato $n + 1$. Questa è solo l'intuizione dell'induzione ben fondata utilizzata nel metodo delle asserzioni induttive.

- **Funzione parzialmente corretta:** una funzione è *parzialmente corretta* se per ogni ingresso che soddisfa la pre-condizione, se la funzione termina, allora l'uscita soddisfa la post-condizione⁵. Si dice “*parzialmente*” perché si ammette la terminazione della funzione. Il termine “*corretta*” viene usato perché pre e post-condizione sono valide.
- **Funzione totalmente corretta:** una funzione è *totalmente corretta* se per ogni ingresso che soddisfa la pre-condizione, la funzione termina e l'uscita soddisfa la post-condizione.

⁴program counter.

⁵Nota: stiamo parlando degli ingressi e uscite delle funzioni di un programma, non dell'ingresso e dell'uscita di un programma.

3. **Metodo delle funzioni d'ordine:** viene usata sui valori e sulla ricorsione. Viene usata per la totale correttezza, quindi per dimostrare la terminazione. **Funzione d'ordine** perché tipicamente la terminazione viene dimostrata dando una funzione definita come un'espressione costruita sulle variabili di un programma che mappa l'espressione ad un valore, in un dominio, con un ordinamento ben fondato (per esempio i numeri naturali). Si dimostra che, ad ogni chiamata ricorsiva o ad ogni iterazione/chiamata ricorsiva del ciclo, l'espressione diventa più piccola. In un ordinamento ben fondato, per esempio su i numeri naturali, se si definisce una funzione su un dominio con ordinamento ben fondato in termini di variabili di programma e si mostra che ad ogni iterazione l'espressione diventa più piccola, siccome lo zero è il fondo dell'ordinamento ben fondato, non potrà scendere all'infinito e quindi il ciclo terminerà.

Le specifiche non sono completamente automatizzate mentre i metodi delle asserzioni induttive e delle funzioni d'ordine sì.



Figura 1: compilatore.

In un linguaggio di programmazione, un compilatore (fig. 1) prende in ingresso un programma P in un linguaggio sorgente e restituisce un programma O nel linguaggio oggetto.

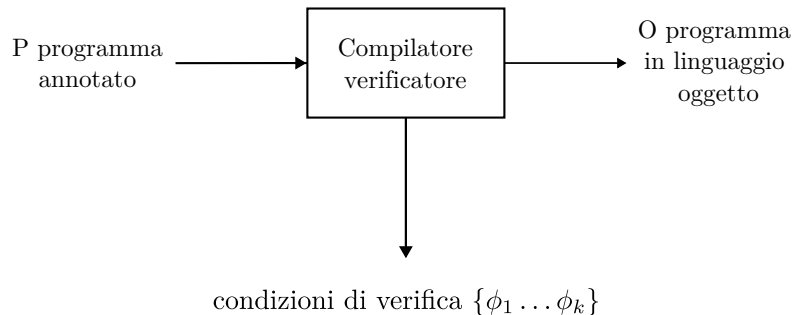


Figura 2: compilatore verificatore.

Un compilatore verificatore (fig. 2) prende in ingresso un programma P annotato (pre-condizioni, post-condizioni, asserzioni e programma nel linguaggio sorgente di prima) e restituisce in uscita:

1. un programma O nel linguaggio oggetto del compilatore precedente;
2. delle formule $\{\phi_1 \dots \phi_k\}$ dette **condizioni di verifica** che sono formule la cui validità assicura che il programma è parzialmente corretto se si usa solo il metodo delle asserzioni induttive e totalmente corretto se si usa anche il metodo delle funzioni d'ordine. Il metodo delle asserzioni induttive e il metodo delle funzioni d'ordine sono *pienamente automatizzabili*.

Questi compilatori implementano i metodi delle asserzioni induttive e delle funzioni d'ordine. Purtroppo non sono completamente automatizzabili come in fig. 3 per il teorema 1.

$\forall i \ 1 \leq i \leq k$ si ha che $\neg\phi_i$ in un dimostratore di teoremi che sia una **procedura di decisione** risponde con:

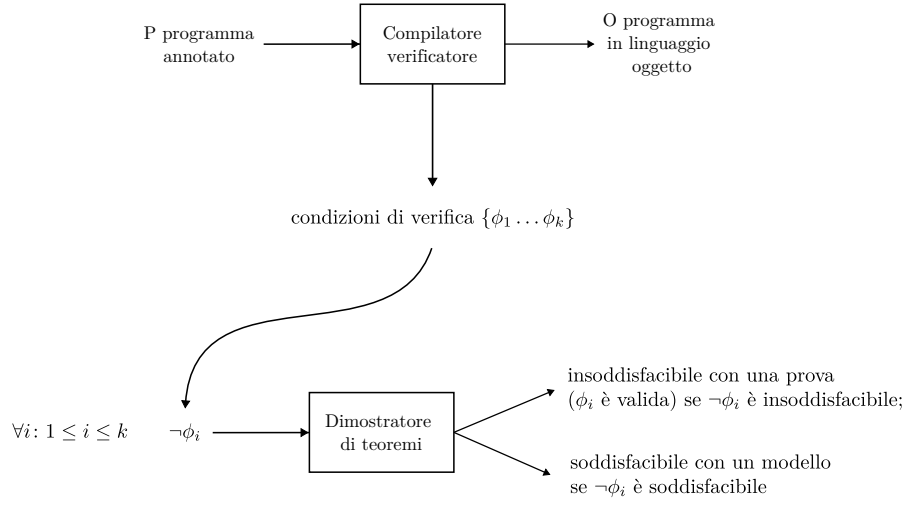


Figura 3: dimostratore di teoremi nel caso ideale.

- insoddisfacibile con una prova (ϕ_i è valida) se $\neg\phi_i$ è insoddisfacibile;
- soddisfacibile con un modello se $\neg\phi_i$ è soddisfacibile.

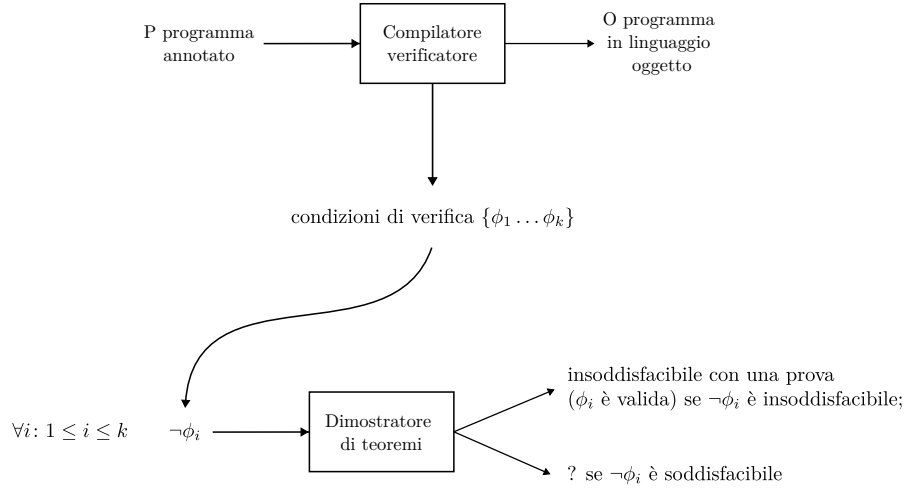


Figura 4: il dimostratore di teoremi nella logica del 1° è semidecidibile.

Nella logica del 1° ordine, l'insoddisfacibilità o equivalentemente la validità è solo semi-decidibile.

Un dimostratore di teoremi che prende in ingresso $\neg\phi_i$ restituisce:

- insoddisfacibile⁶ con prova se $\exists i. \neg\phi_i$ è insoddisfacibile ($\forall i. \phi_i$ è valida⁷);
- ? se $\neg\phi_i$ è soddisfacibile.

⁶significa che è sempre falsa.

⁷è sempre vera.

Se $\neg\phi_i$ è non soddisfacibile, vuol dire che ϕ_i è in-valida (cioè non valida). Quindi si può ricostruire lo stato di non validità del programma (quando viene restituito il modello).

Riferimenti bibliografici

- [itf] it.frwiki.wiki. *Teorema di Rice*. URL: https://it.frwiki.wiki/wiki/Th%C3%A9or%C3%A8me_de_Rice.
- [Wik] Wikipedia. *Teorema di Rice*. URL: https://it.wikipedia.org/wiki/Teorema_di_Rice.